



Programação Orientada a Objetos com Java

Classes

Objetos

Atributos

Métodos

Encapsulamento

Herança

Polimorfismo

Projeto final

Objetivos do curso

- Ao final, você será capaz de **modelar problemas reais em Java** com POO.
- **Escrever classes** com atributos e métodos bem projetados.
- **Criar e usar objetos** (instâncias) com confiança.
- **Proteger dados** com encapsulamento (private, getters e setters).
- **Reutilizar código** com construtores, sobrecarga e herança.
- **Construir um projeto completo** (menu, cadastros, listas e buscas).
- Desenvolver o **pensamento orientado a objetos**, não só a sintaxe.

Módulo 0 · 4 horas

Primeiros passos com Java

Ambiente pronto, primeiros programas e a base da linguagem

-
- ✓ Instalar o JDK e uma IDE
 - ✓ Escrever e executar o primeiro programa
 - ✓ Usar variáveis, tipos e operadores
 - ✓ Fazer decisões (if/else) e repetições (for/while)
 - ✓ Trabalhar com vetores (arrays)

Agenda do módulo

1. O que é programar?

- O que é um programa, linguagens, por que Java

2. Como o Java funciona

- JVM, JDK, JRE e compilação

3. Instalação e ambiente

- JDK + IDE, primeiro programa

4. Fundamentos da linguagem

- Variáveis, tipos, operadores, entrada do usuário

5. Controle de fluxo

- if/else, for, while, arrays

6. Exercícios e resumo

- 3 exercícios práticos + pontos-chave

O que é programação?

- **Programar** é dar instruções claras e precisas para o computador executar.
- O computador faz exatamente o que mandamos — por isso as instruções precisam ser **sem ambiguidade**.
- Programas estão em toda parte: celular, banco, jogos, carros, redes sociais.
- **Algoritmo**: a sequência lógica de passos para resolver um problema.
- Analogia: programar é escrever uma **receita de bolo** muito detalhada.
 - Ingredientes → **dados** (variáveis)
 - Passos da receita → **instruções** (código)
 - Bolo pronto → **resultado** (saída do programa)

O que é um programa de computador?

- Um **programa** é um conjunto de instruções escritas em uma linguagem de programação.
- O computador **não entende português nem inglês** — ele entende apenas **0 e 1** (binário).
- Usamos **linguagens de programação** como ponte entre o humano e a máquina.
- O **código-fonte** é o texto que escrevemos; o **programa executável** é o que a máquina roda.
- Exemplos de linguagens: Java, Python, C, JavaScript, C#, PHP...
- Cada linguagem tem regras próprias de escrita: a **sintaxe**.
- Erro de sintaxe = frase fora das regras; o computador não consegue interpretar.

Linguagens de programação: compiladas e interpretadas

- **Compilada:** o código-fonte é traduzido uma vez para a linguagem da máquina.
 - Exemplo: C, C++ e também **Java** (com um detalhe especial).
- **Interpretada:** o código é lido e executado linha por linha na hora.
 - Exemplo: Python, JavaScript.
- **Java é compilado + interpretado:**
 - O compilador **javac** transforma o código em **bytecode** (intermediário).
 - A **JVM** (máquina virtual) interpreta o bytecode em qualquer sistema.
- Vantagem prática: **escreva uma vez, rode em qualquer lugar.**

Por que aprender Java?

- Uma das linguagens **mais usadas no mercado** há mais de 20 anos.
- **Multiplataforma:** roda em Windows, Linux, macOS, celulares e servidores.
- Base sólida de **POO**: o Java foi criado para ensinar e praticar o paradigma.
- **Forte tipagem:** erros são detectados cedo, ideal para iniciantes.
- Imensa **biblioteca padrão** (coleções, arquivos, redes, interfaces gráficas).
- Ecossistema gigante: Android, sistemas bancários, grandes empresas.
- Comunidade enorme: dúvidas resolvidas em segundos na internet.

JDK, JRE e JVM: o que é cada um?

Sigla	Nome	O que faz	Preciso para...
JDK	Java Development Kit	Ferramentas para DESENVOLVER: compilador <code>javac</code> , bibliotecas	criar programas
JRE	Java Runtime Environment	Ambiente para EXECUTAR programas Java	rodar programas
JVM	Java Virtual Machine	Máquina virtual que roda o bytecode em cada sistema	tudo rodar

Do código ao programa: o caminho do Java

O mesmo .class roda no Windows, no Linux e no Mac — graças à JVM.

fluxo.txt

```
/* 1. Você escreve o código-fonte */  
OlaMundo.java  
  
/* 2. O compilador traduz para bytecode */  
javac OlaMundo.java -> OlaMundo.class  
  
/* 3. A JVM executa o bytecode em qualquer sistema */  
java OlaMundo -> Olá, mundo!
```

Instalação: passo a passo (Windows / Linux / Mac)

- **Passo 1 — Baixe o JDK** (versão LTS mais recente, ex.: 21) no site da Oracle ou em distribuições OpenJDK (Adoptium/Temurin).
- **Passo 2 — Instale** seguindo o instalador padrão.
- **Passo 3 — Confirme** que funcionou: abra o terminal e digite:
 - `java -version` → mostra a versão instalada
 - `javac -version` → mostra o compilador
- Se o comando não for encontrado, configure a **variável de ambiente** JAVA_HOME/PATH.
- **Passo 4 — Instale uma IDE** (IntelliJ IDEA Community, VS Code ou Eclipse) — próxima aula.
- Dúvida? O site oficial java.com e as comunidades brasileiras ajudam rapidinho.

Escolhendo a IDE (ambiente de desenvolvimento)

- A IDE é o 'editor turbinado' onde escrevemos código: **destaque de sintaxe, sugestões e botão de executar**.
- **IntelliJ IDEA Community** — a mais popular para Java, gratuita. Recomendada para o curso.
- **Eclipse** — clássica e gratuita, muito usada em universidades.
- **VS Code** — leve, com extensão 'Extension Pack for Java'.
- Todas servem: o código que escreveremos **funciona igual** em qualquer uma.
- No IntelliJ: **New Project** → **Java** → criar classe → clicar no botão ► para executar.
- Terminal também vale: o Java roda direto no prompt de comando.

Seu primeiro programa: Olá, Mundo!

OlaMundo.java

```
public class OlaMundo {  
    public static void main(String[] args) {  
        System.out.println("Olá, mundo!");  
    }  
}
```

Saída

Olá, mundo!

- Todo programa Java precisa de uma **classe** (aqui: `OlaMundo`).
- O **método** `main` é o ponto de partida: o Java começa a executar por ele.
- `System.out.println(...)` imprime uma linha no console.

Entendendo o Olá, Mundo linha por linha

OlaMundo.java

```
public class OlaMundo {           // 1. declara a classe
    public static void main(String[] args) { // 2. ponto de entrada
        // 3. instrução: imprime texto e pula a linha
        System.out.println("Olá, mundo!");
    } // 4. fim do método main
} // 5. fim da classe
```

- `public` → pode ser acessado por qualquer um; `class` → é uma classe.
- `static void main` → método que não devolve valor; executa sem criar objeto.
- `String[] args` → parâmetros recebidos do terminal (veremos depois).
- `;` encerra cada instrução — **não esqueça!**

Compilando e executando pelo terminal

Na IDE isso acontece em 1 clique; no terminal são 2 comandos.

Terminal

```
$ javac OlaMundo.java      # compila: gera OlaMundo.class
$ java OlaMundo            # executa a classe (sem o .java)
Olá, mundo!
```

- `javac` = compilador. `java` = JVM executando a classe.
- Na IDE, pressione **Shift+F10** (IntelliJ) ou o botão ▶.

Variáveis: guardando informações

● ● ● Variaveis.java

```
public class Variaveis {  
    public static void main(String[] args) {  
        int idade = 25;           // número inteiro  
        double altura = 1.78;     // número decimal  
        boolean estudante = true; // verdadeiro ou falso  
        char inicial = 'J';       // um único caractere  
        String nome = "João";     // texto (classe do Java)  
  
        System.out.println(nome + " tem " + idade + " anos.");  
    }  
}
```

Saída

João tem 25 anos.

- Padrão: `tipo nome = valor;` — primeiro o tipo, depois o nome.
- `String` é a única **com inicial maiúscula** (é uma classe, não um tipo primitivo).
- O operador `+` junta texto e variáveis (concatenação).

Tipos primitivos do Java

Tipo	O que guarda	Exemplo	Valor padrão
int	números inteiros	int idade = 25;	0
double	números decimais	double preco = 19.90;	0.0
boolean	verdadeiro ou falso	boolean ativo = true;	false
char	um único caractere	char letra = 'A';	'\u0000'
String	texto (classe, não primitivo)	String nome = "Ana";	null

Exemplo: variáveis em ação

Uma conta simples no banco:

Conta.java

```
public class Conta {  
    public static void main(String[] args) {  
        String titular = "Maria";  
        double saldo = 1000.00;  
        double saque = 250.50;  
        saldo = saldo - saque;        // atualiza o saldo  
        System.out.println("Titular: " + titular);  
        System.out.println("Saldo após o saque: R$ " + saldo);  
    }  
}
```

Saída

Titular: Maria
Saldo após o saque: R\$ 749.5

- Observe: `saldo = saldo - saque;` lê o valor antigo, subtrai e guarda o novo na mesma variável.

Operadores que você precisa conhecer

Categoria	Operadores	Exemplo	Resultado
Aritméticos	+ - * / % (resto)	10 % 3	1
Comparação	== != > < >= <=	5 > 3	true
Lógicos	&& (e) (ou) ! (não)	true && false	false
Atribuição	= += -= *= /=	int x = 5; x += 2;	x = 7

Lendo dados do teclado: a classe Scanner

● ● ● Saudacao.java

```
import java.util.Scanner;          // importa a classe do teclado

public class Saudacao {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Qual é o seu nome? ");
        String nome = teclado.nextLine(); // lê uma linha
        System.out.print("Qual a sua idade? ");
        int idade = teclado.nextInt();    // lê um inteiro

        System.out.println(nome + " tem " + idade + " anos.");
        teclado.close();
    }
}
```

Saída

```
Qual é o seu nome? Ana
Qual a sua idade? 22
Ana tem 22 anos.
```

- `import` no topo traz classes prontas do Java.
- `nextLine()` lê texto, `nextInt()` lê inteiro, `nextDouble()` lê decimal.
- `System.out.print` não pula a linha; `println` pula.

Exemplo: programa de decisão (if / else)

A escola aprova quem tira nota maior ou igual a 6:

● ● ● Aprovacao.java

```
public class Aprovacao {  
    public static void main(String[] args) {  
        double nota = 7.5;  
        if (nota >= 6) {  
            System.out.println("Aprovado!");  
        } else {  
            System.out.println("Reprovado.");  
        }  
    }  
}
```

Saída

Aprovado!

- Estrutura: `if (condição) { ... } else { ... }`. A condição precisa ser `boolean`.

Repetição: for e while

● ● ● Repeticao.java

```
public class Repeticao {  
    public static void main(String[] args) {  
        // for: sabemos quantas vezes repetir  
        for (int i = 1; i <= 3; i++) {  
            System.out.println("Volta " + i);  
        }  
        // while: repetimos enquanto a condição for verdadeira  
        int tentativa = 1;  
        while (tentativa <= 3) {  
            System.out.println("Tentativa " + tentativa);  
            tentativa++;  
        }  
    }  
}
```

Saída

```
Volta 1  
Volta 2  
Volta 3  
Tentativa 1  
Tentativa 2  
Tentativa 3
```

- `i++` é a mesma coisa que `i = i + 1`.
- Cuidado com o laço infinito: se a condição nunca ficar falsa... o programa nunca para!

Vetores (arrays): guardando várias informações

Vetores.java

```
public class Vetores {  
    public static void main(String[] args) {  
        double[] notas = {8.0, 9.0, 7.0}; // 3 posições (0, 1 e 2)  
        double soma = 0;  
        for (int i = 0; i < notas.length; i++) {  
            soma = soma + notas[i];  
        }  
        System.out.println("Média: " + (soma / notas.length));  
    }  
}
```

Saída

Média: 8.0

- A primeira posição é a **0** (índice 0).
- `notas.length` → tamanho do vetor (aqui, 3).
- No Módulo 3 veremos coleções melhores: `ArrayList`.

Exercício 1 — Calculadora de idade em dias

- Crie um programa que pergunta a **idade** do usuário com `Scanner`.
- Calcule quantos **dias** essa pessoa já viveu (aprox.: idade × 365).
- Mostre: "**Você já viveu aproximadamente X dias.**"
- Teste com sua própria idade e confira o resultado.

Dica: Use `int idade = teclado.nextInt();` e multiplique por 365.

Resolução — Exercício 1 (Calculadora de idade em dias)

● ● ● DiasDeVida.java

```
import java.util.Scanner;

public class DiasDeVida {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        System.out.print("Qual a sua idade? ");
        int idade = teclado.nextInt();
        int dias = idade * 365;
        System.out.println("Você já viveu aproximadamente " + dias + " dias.");
        teclado.close();
    }
}
```

Saída

Qual a sua idade? 20
Você já viveu aproximadamente 7300 dias.

Exercício 2 — Par ou ímpar

- Peça um número inteiro ao usuário.
- Use o operador % (resto) para descobrir se é par ou ímpar.
- Exiba "**O número X é par.**" ou "**O número X é ímpar.**"
- Desafio: se for par e maior que 100, exiba também "Número grande!".**

Dica: Um número é par quando `numero % 2 == 0`.

Resolução — Exercício 2 (Par ou ímpar)

● ● ● ParImpar.java

```
import java.util.Scanner;

public class ParImpar {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        System.out.print("Digite um número: ");
        int n = teclado.nextInt();
        if (n % 2 == 0) {
            System.out.println("O número " + n + " é par.");
            if (n > 100) {
                System.out.println("Número grande!");
            }
        } else {
            System.out.println("O número " + n + " é ímpar.");
        }
        teclado.close();
    }
}
```

Saída

Digite um número: 150
O número 150 é par.
Número grande!

Exercício 3 — Soma dos pares de 1 a 100

- Use um laço `for` para percorrer os números de 1 a 100.
- Some **apenas os números pares**.
- Ao final, exiba o total.
- Depois, refaça com `while` — o resultado deve ser o mesmo!

Dica: Dentro do laço, use `if (i % 2 == 0)` para somar só os pares.

Resolução — Exercício 3 (Soma dos pares de 1 a 100)

● ● ● SomaPares.java

```
public class SomaPares {  
    public static void main(String[] args) {  
        int soma = 0;  
        for (int i = 1; i <= 100; i++) {  
            if (i % 2 == 0) {  
                soma = soma + i;  
            }  
        }  
        System.out.println("Soma dos pares de 1 a 100: " + soma);  
    }  
}
```

Saída

Soma dos pares de 1 a 100: 2550

Resumo do Módulo 0

- Programar é **dar instruções claras**; um **algoritmo** é a sequência de passos.
- Java = **compilado para bytecode** + executado pela **JVM** em qualquer sistema.
- **JDK** para desenvolver, **JRE** para rodar, **JVM** para executar.
- Primeiro programa: `class` + `main` + `System.out.println`.
- Variáveis guardam dados: `int`, `double`, `boolean`, `char`, `String`.
- `if/else` decide; `for/while` repetem; `arrays` guardam várias coisas.
- `Scanner` lê o que o usuário digita.

Checkpoint rápido — Módulo 0

1. Qual comando compila um arquivo Java?

- (A) java OlaMundo
- (B) javac OlaMundo.java
- (C) run OlaMundo

2. O que a JVM faz?

- (A) Escreve o código por você
- (B) Executa o bytecode em qualquer sistema
- (C) Apenas edita textos

3. Qual é o valor padrão de uma variável `int` sem valor?

- (A) null
- (B) 1
- (C) 0

Gabarito: 1→B, 2→B, 3→C

De olho no próximo passo: por que POO?

- Até aqui, guardamos dados em **variáveis soltas** e **vetores**.
- Em um sistema real (escola, banco, loja) isso vira um caos:
 - nome, nota, matricula de **cada aluno** ficam em vetores separados.
 - A lógica (calcular média, aprovar) fica espalhada pelo programa.
- A **Programação Orientada a Objetos (POO)** junta:
 - **dados** (atributos) e **comportamentos** (métodos) de um conceito em uma única peça: a **classe**.
- É assim que o Java foi desenhado — e é o que vamos aprender agora!

Módulo 1 · 6 horas

Classes e Objetos

Os dois conceitos mais importantes da POO — com muitos exemplos

-
- ✓ Entender o que são classes e objetos (e a diferença entre eles)
 - ✓ Criar classes com atributos e métodos
 - ✓ Instanciar objetos com o operador new
 - ✓ Usar métodos com retorno e parâmetros
 - ✓ Resolver 3 exercícios práticos completos

Agenda do módulo

1. O problema que a POO resolve

- Cadastro de alunos sem POO × com POO

2. Conceitos centrais

- Objeto, classe, analogias, classe × objeto

3. Na prática

- Criar classe Pessoa, usar new, acessar atributos

4. Métodos

- Sem retorno, com retorno, com parâmetros, this

5. Exemplos completos

- Classe Carro passo a passo

6. Exercícios + resumo

- Livro, Cachorro, Retângulo + quiz

O problema: cadastro de alunos sem POO

- Queremos cadastrar **3 alunos** com nome, matrícula e nota.
- Sem POO, usaríamos variáveis e vetores separados:
 - `String[] nomes = {"Ana", "Beto", "Carla"};`
 - `int[] matriculas = {101, 102, 103};`
 - `double[] notas = {8.5, 7.0, 9.5};`
- E se fossem **3.000 alunos**? Vetores gigantes e fáceis de bagunçar.
- E se cada aluno tiver **endereço, telefone, email, foto**? Vetores demais!
- Os **dados estão espalhados** e a lógica também — um caos para manter.

A ideia da POO: juntar dados + comportamento

- A POO **agrupa em uma única peça** tudo o que pertence a um conceito:
 - **atributos** → as características (dados): nome, matrícula, nota.
 - **métodos** → os comportamentos (ações): calcularMedia, exibirDados.
- Essa peça é a **classe** ``Aluno``; cada aluno real é um **objeto** dessa classe.
- Vetor de 3.000 alunos? Simples: `ArrayList<Aluno>` (Módulo 3).
- O programa passa a pensar em termos do **mundo real**: aluno, carro, conta bancária.
- É isso que a POO faz: **modelar o mundo em código**.

O que é um objeto?

- **Objeto** = uma 'coisa' concreta do mundo real representada no programa.
- Todo objeto tem:
 - **Estado** → os valores dos seus atributos naquele momento.
 - **Comportamento** → o que ele sabe fazer (métodos).
- Exemplos:
 - **O seu cachorro** → estado: nome, idade, raça · comportamento: latir, correr.
 - **O seu carro** → estado: marca, modelo, velocidade · comportamento: acelerar, frear.
 - **Uma conta bancária** → estado: titular, saldo · comportamento: depositar, sacar.
- Em Java, um objeto é criado em **tempo de execução** dentro da memória.

O que é uma classe?

- **Classe** = o **molde** / a **planta** que define como os objetos serão.
- A classe **não é o objeto** — ela descreve quais atributos e métodos os objetos terão.
- A partir de **uma** classe podemos criar **milhares** de objetos.
- Analogia: a classe é a **forma de biscoito**; os objetos são os **biscoitos**.
 - Mesma forma, biscoitos diferentes (valores diferentes).
- Analogia: a classe é a **planta da casa**; os objetos são as **casas construídas**.
- Quando criamos um objeto a partir da classe, dizemos que **instanciamos** a classe.

Classe x Objeto

CLASSE (molde)

- É a **definição** — o molde.
- Descreve atributos e métodos.
- Existe **uma única vez** no código.
- Não ocupa memória para dados.
- Ex.: `class Cachorro`

OBJETO (instância)

- É o **resultado concreto** — o biscoito.
- Tem valores reais nos atributos.
- Podem existir **vários** iguais.
- Ocupa memória em tempo de execução.
- Ex.: `rex = new Cachorro()`

Anatomia de uma classe em Java

```
● ● ● Pessoa.java

public class Pessoa {      // 1. palavra-chave class + nome
    // 2. ATRIBUTOS: as características
    String nome;
    int idade;

    // 3. MÉTODOS: os comportamentos (por enquanto, vazio)
}
```

- O nome da classe usa **PascalCase** (*Pessoa*, *ContaBancaria*).
- Por enquanto a classe só tem atributos — ela é o molde.
- O arquivo deve ter o mesmo nome da classe: *Pessoa.java*.

Criando objetos: o operador new

TestePessoa.java

```
public class TestePessoa {  
    public static void main(String[] args) {  
        // 1. DECLARA a variável do tipo Pessoa  
        Pessoa joao;  
        // 2. CRIA o objeto na memória (new) e o associa à variável  
        joao = new Pessoa();  
  
        // 3. Preenche os atributos do objeto  
        joao.nome = "João";  
        joao.idade = 25;  
    }  
}
```

- `new Pessoa()` **reserva memória** e monta o objeto.
- A variável `joao` é uma **referência**: aponta para o objeto na memória.
- Acessamos os atributos com **ponto**: `joao.nome`.

Exemplo: Pessoa na prática

Criamos a classe, criamos 1 objeto e mostramos os dados:

TestePessoa.java

```
public class TestePessoa {  
    public static void main(String[] args) {  
        Pessoa joao = new Pessoa();  
        joao.nome = "João";  
        joao.idade = 25;  
  
        System.out.println("Nome: " + joao.nome);  
        System.out.println("Idade: " + joao.idade);  
    }  
}
```

Saída

Nome: João
Idade: 25

- Pessoa.java (molde) + TestePessoa.java (uso). Em projetos reais, cada classe fica em seu próprio arquivo.

Exemplo: vários objetos independentes

Uma classe, dois objetos — cada um com seus próprios valores:

TestePessoa.java

```
public class TestePessoa {  
    public static void main(String[] args) {  
        Pessoa ana = new Pessoa();  
        ana.nome = "Ana";  
        ana.idade = 22;  
  
        Pessoa beto = new Pessoa();  
        beto.nome = "Beto";  
        beto.idade = 30;  
  
        System.out.println(ana.nome + " tem " + ana.idade + " anos.");  
        System.out.println(beto.nome + " tem " + beto.idade + " anos.");  
        System.out.println("Objetos são independentes entre si.");  
    }  
}
```

Saída

```
Ana tem 22 anos.  
Beto tem 30 anos.  
Objetos são independentes entre si.
```

- Mudar `ana.idade` não afeta `beto.idade`: cada objeto tem sua própria cópia dos atributos.

Valores padrão dos atributos

Tipo do atributo	Valor inicial	Significado
int	0	zero
double	0.0	zero decimal
boolean	false	falso
char	'\u0000'	caractere vazio
String	null	ainda não aponta para nenhum texto

Atributos: o estado do objeto

- **Atributo** = variável que pertence ao objeto (a característica).
- Declarados dentro da classe, fora dos métodos.
- Cada objeto tem **sua própria cópia** dos atributos.
- Tipos podem ser primitivos (`int`, `double`, `boolean`) ou classes (`String`).
- Podemos ter quantos quisermos: nome, cor, preço, estoque, fabricante...
- Exemplo de classe com vários atributos:
 - `class Produto { String nome; double preco; int quantidadeEmEstoque; }`
- Regra de nome: **camelCase** — `quantidadeEmEstoque`, `precoUnitario`.

Métodos: o comportamento do objeto

- **Método** = bloco de código com um nome, que executa uma ação.
- São as 'habilidades' do objeto: `acelerar()`, `calcularMedia()`, `exibir()`.
- Estrutura básica:
 - `tipoDeRetorno nomeDoMetodo(parâmetros) { ... }`
- **Retorno** = o que o método devolve (`void` = nada).
- **Parâmetros** = informações que o método recebe para trabalhar.
- Método simples (sem retorno e sem parâmetros):
 - `void exibir() { System.out.println("Olá!"); }`
- Para chamar: `joao.exibir();` — sempre com o ponto.

Anatomia de um método, parte por parte

```
Pessoa.java

public class Pessoa {
    String nome;
    int idade;

    // modificador  retorno  nome      parâmetros
    public void cumprimentar() {
        System.out.println("Olá, eu sou " + nome);
    }

    public int calcularIdadeEmMeses() { // retorna int
        return idade * 12;             // return devolve o valor
    }
}
```

- `void` = método **não devolve** nada.
- `int` = método **devolve um int** — usa `return`.
- Dentro do método, `nome` e `idade` se referem ao **próprio objeto**.

Exemplo: método que devolve um valor

A classe *Pessoa* ganha o método *calcularIdadeEmMeses*:

TestePessoa.java

```
public class TestePessoa {  
    public static void main(String[] args) {  
        Pessoa maria = new Pessoa();  
        maria.nome = "Maria";  
        maria.idade = 30;  
  
        int meses = maria.calcularIdadeEmMeses();  
        System.out.println(maria.nome + " tem " + meses + " meses de vida.");  
    }  
}
```

Saída

Maria tem 360 meses de vida.

- O valor devolvido por `return` é **capturado** na variável `meses` (ou usado direto).

Exemplo: métodos com parâmetros

Agora o objeto recebe informações para trabalhar:

```
Pessoa.java

public class Pessoa {
    String nome;
    int idade;

    void fazerAniversario() {
        idade = idade + 1;
    }

    void mudarNome(String novoNome) {    // parâmetro novoNome
        nome = novoNome;                 // aplica o valor recebido
    }
}
```

Saída

Antes: 25 anos
Depois do aniversário: 26 anos

- Exercício mental: o que `fazerAniversario()` muda no estado do objeto?

this: o objeto falando de si mesmo

● ● ● Pessoa.java

```
public class Pessoa {  
    String nome;  
    int idade;  
  
    void cadastrar(String nome, int idade) {  
        // this.nome = ATRIBUTO do objeto  
        // nome      = parâmetro recebido  
        this.nome = nome;  
        this.idade = idade;  
    }  
  
    void exibir() {  
        System.out.println(nome + ", " + idade + " anos");  
    }  
}
```

- Quando o nome do parâmetro **é igual** ao do atributo, usamos `this.` para diferenciar.
- `this` aponta para o **próprio objeto** que está executando o método.
- Regra prática: se houver conflito de nomes, use `this.atributo`.

Exemplo completo: classe Carro

Vamos modelar um carro: marca, modelo, velocidade e as ações acelerar/frear/exibir:

```
Carro.java

public class Carro {
    String marca;
    String modelo;
    int velocidade;

    void acelerar(int quantidade) {
        velocidade = velocidade + quantidade;
    }

    void frear(int quantidade) {
        velocidade = velocidade - quantidade;
        if (velocidade < 0) {
            velocidade = 0;           // nunca fica negativa
        }
    }

    void exibir() {
        System.out.println(modelo + " está a " + velocidade + " km/h");
    }
}
```

- Atributos: `marca`, `modelo`, `velocidade`. Métodos: `acelerar`, `frear`, `exibir`.

Exemplo: usando o Carro no programa principal

Criamos o objeto, chamamos seus métodos e observamos o estado mudar:

TesteCarro.java

```
public class TesteCarro {  
    public static void main(String[] args) {  
        Carro fusca = new Carro();  
        fusca.marca = "Volkswagen";  
        fusca.modelo = "Fusca";  
  
        fusca.acelerar(40);  
        fusca.exibir();  
        fusca.frear(10);  
        fusca.exibir();  
        fusca.frear(100);           // vai tentar ficar negativa...  
        fusca.exibir();  
    }  
}
```

Saída

```
Fusca está a 40 km/h  
Fusca está a 30 km/h  
Fusca está a 0 km/h
```

- A proteção `if (velocidade < 0)` evita estado inválido — esse cuidado será tema do Módulo 2 (encapsulamento).

Exercício 1 — Classe Livro

- Crie a classe `Livro` com atributos: `titulo`, `autor`, `paginas`.
- Adicione o método `void exibir()` que mostra: "**Título: ... · Autor: ... · X páginas**".
- No `main`, crie **2 livros**, preencha os dados e chame `exibir()` dos dois.
- Teste: troque o título de um deles e veja que o outro não muda.

Dica: Lembre: atributos dentro da classe, preenchimento com `objeto.atributo = valor;` no main.

Resolução — Exercício 1 (Classe Livro)

● ● ● Livro.java

```
public class Livro {  
    String titulo;  
    String autor;  
    int paginas;  
  
    void exibir() {  
        System.out.println("Título: " + titulo + " · Autor: " + autor  
            + " · " + paginas + " páginas");  
    }  
}
```

Resolução — Exercício 1 (Classe Livro — programa principal)

TesteLivro.java

```
public class TesteLivro {  
    public static void main(String[] args) {  
        Livro livro1 = new Livro();  
        livro1.titulo = "Dom Casmurro";  
        livro1.autor = "Machado de Assis";  
        livro1.paginas = 256;  
  
        Livro livro2 = new Livro();  
        livro2.titulo = "O Pequeno Príncipe";  
        livro2.autor = "Antoine de Saint-Exupéry";  
        livro2.paginas = 96;  
  
        livro1.exibir();  
        livro2.exibir();  
    }  
}
```

Saída

Título: Dom Casmurro · Autor: Machado de Assis · 256 páginas

Título: O Pequeno Príncipe · Autor: Antoine de Saint-Exupéry · 96 páginas

Exercício 2 — Classe Cachorro

- Crie a classe `Cachorro` com atributos: `nome`, `idade`, `raca`.
- Método `void latir()` → imprime **"Au au!"**.
- Método `void correr(int metros)` → imprime **"[nome] correu X metros"**.
- No `main`, crie um cachorro, preencha e faça-o latir e correr 30 metros.

Dica: Dentro de `correr`, use o atributo `nome` para personalizar a mensagem.

Resolução — Exercício 2 (Classe Cachorro)

● ● ● Cachorro.java

```
public class Cachorro {  
    String nome;  
    int idade;  
    String raca;  
  
    void latir() {  
        System.out.println("Au au!");  
    }  
  
    void correr(int metros) {  
        System.out.println(nome + " correu " + metros + " metros.");  
    }  
}
```

Resolução — Exercício 2 (Classe Cachorro — programa principal)

TesteCachorro.java

```
public class TesteCachorro {  
    public static void main(String[] args) {  
        Cachorro rex = new Cachorro();  
        rex.nome = "Rex";  
        rex.idade = 3;  
        rex.raca = "Labrador";  
  
        rex.latir();  
        rex.correr(30);  
    }  
}
```

Saída

Au au!
Rex correu 30 metros.

Exercício 3 — Classe Retângulo

- Crie a classe `Retangulo` com atributos: `largura` e `altura` (double).
- Método `double calcularArea()` → retorna `largura × altura`.
- Método `double calcularPerimetro()` → retorna `2 × (largura + altura)`.
- No `main`, crie um retângulo `5 × 3` e exiba área e perímetro.
- Desafio: crie um método `void redimensionar(double fator)` que multiplica os dois lados.

Dica: Método com retorno usa `return`; capture o valor em uma variável no `main`.

Resolução — Exercício 3 (Classe Retângulo)

Retangulo.java

```
public class Retangulo {  
    double largura;  
    double altura;  
  
    double calcularArea() {  
        return largura * altura;  
    }  
  
    double calcularPerimetro() {  
        return 2 * (largura + altura);  
    }  
  
    void redimensionar(double fator) {  
        largura = largura * fator;  
        altura = altura * fator;  
    }  
}
```

Resolução — Exercício 3 (Classe Retângulo — programa principal)

TesteRetangulo.java

```
public class TesteRetangulo {  
    public static void main(String[] args) {  
        Retangulo r = new Retangulo();  
        r.largura = 5;  
        r.altura = 3;  
  
        System.out.println("Área: " + r.calcularArea());  
        System.out.println("Perímetro: " + r.calcularPerimetro());  
  
        r.redimensionar(2);  
        System.out.println("Nova área: " + r.calcularArea());  
    }  
}
```

Saída

```
Área: 15.0  
Perímetro: 16.0  
Nova área: 60.0
```

Erros comuns de quem está começando

Erros.java

```
// ERRO 1: esquecer o new – 'null' ainda não aponta para objeto
Pessoa joao;
joao.nome = "João"; // NullPointerException!

// ERRO 2: esquecer o return em método que devolve valor
int calcular() {
    int x = 10;
    // falta 'return x;' -> erro de compilação
}

// ERRO 3: nome da classe diferente do arquivo
// arquivo: Teste.java -> public class Teste (ok)
// arquivo: Teste.java -> public class Tst (erro!)
```

- **NullPointerException**: você usou um objeto que ainda não foi criado com `new`.
- **Método deve retornar um valor**: todo caminho precisa de `return`.
- **Arquivo x classe**: o nome público deve bater com o nome do arquivo.

Regras de nomenclatura (convenções Java)

- **Classes:** PascalCase — `ContaBancaria`, `Aluno`, `CadastroProdutos`.
- **Atributos e métodos:** camelCase — `saldo`, `calcularMedia`, `quantidadeEmEstoque`.
- **Constantes:** letras maiúsculas com `_` — `TAXA_JUROS` (veremos no Módulo 2).
- Nomes devem ser **significativos**: `preco` em vez de `p`, `calcularMedia` em vez de `cm`.
- Ações geralmente são **verbos**: `acelerar()`, `exibir()`, `cadastrar()`.
- Seguir convenções torna o código **legível para outras pessoas** (e para você no futuro).

Resumo do Módulo 1

- **Objeto**: coisa concreta com **estado** (atributos) e **comportamento** (métodos).
- **Classe**: o molde que define atributos e métodos dos objetos.
- **Instanciar** = criar objeto com `new` → `Pessoa p = new Pessoa();`
- Atributos são **variáveis do objeto**; métodos são **ações do objeto**.
- `void` = sem retorno; com `return` o método **devolve um valor**.
- Parâmetros **alimentam** os métodos; `this` resolve conflito de nomes.
- Cada objeto é **independente**: alterar um não altera os outros.

Checkpoint rápido — Módulo 1

1. O que é uma classe?

- (A) Um objeto criado na memória
- (B) O molde que define atributos e métodos
- (C) Um tipo primitivo do Java

2. Qual comando cria um objeto de verdade?

- (A) Pessoa joao;
- (B) new Pessoa()
- (C) Pessoa()

3. Um método declarado `void` deve:

- (A) Devolver um int
- (B) Não devolver nada
- (C) Ter parâmetros obrigatórios

Gabarito: 1→B, 2→B, 3→B

Módulo 2 · 8 horas

Encapsulamento, Construtores e Sobrecarga

Deixe suas classes seguras, flexíveis e profissionais

-
- ✓ Proteger atributos com private (encapsulamento)
 - ✓ Criar getters e setters com validação
 - ✓ Construir objetos com construtores (e sobrecarga)
 - ✓ Sobrecarregar métodos
 - ✓ Usar atributos e métodos estáticos (static)
 - ✓ Entender toString, equals e objetos como parâmetros

Agenda do módulo

1. Encapsulamento

- O problema do atributo público, private, getters/setters

2. Exemplo ContaBancaria

- Regras de negócio protegidas por métodos

3. Construtores

- Construtor padrão, com parâmetros, sobrecarga, this()

4. Sobrecarga de métodos

- Mesmo nome, assinaturas diferentes

5. Static e final

- Atributos de classe, constantes, métodos estáticos

6. Objetos conversando

- Objetos como parâmetros, == vs equals, toString

7. Exercícios + resumo

- Produto, Funcionário, ContaCorrente + quiz

O problema: atributo público desprotegido

Imagine que qualquer código possa alterar o saldo diretamente...

```
ContaBancaria.java  
  
public class ContaBancaria {  
    double saldo;        // público? (na verdade: acesso padrão)  
}
```

Saída

Saldo virou -500.0 ← ESTADO INVÁLIDO!

- Qualquer parte do programa pode fazer `conta.saldo = -500;`.
- Uma conta **nunca** deveria ficar negativa assim.
- O objeto **perdeu o controle** sobre o próprio estado.
- A solução: **encapsulamento** — esconder e controlar o acesso.

Encapsulamento: o princípio

- **Encapsulamento** = esconder os detalhes internos e **controlar o acesso** aos dados.
- Três objetivos principais:
 - **Proteger** o estado (evitar valores inválidos).
 - **Ocultar** como as coisas funcionam (simplificar o uso).
 - **Permitir mudanças** internas sem quebrar quem usa a classe.
- Na prática, em Java:
 - atributos ficam `private` (ninguém de fora acessa direto).
 - métodos `get`/`set` (ou outros métodos) fazem o acesso com regras.
- É como um caixa eletrônico: você usa as funções, não abre a máquina.

Modificadores de acesso

Modificador	Mesma classe	Mesmo pacote	Subclasse	Qualquer lugar
private	✓	—	—	—
(padrão)	✓	✓	—	—
protected	✓	✓	✓	—
public	✓	✓	✓	✓

Aplicando private + getters e setters

```
Pessoa.java

public class Pessoa {
    private String nome;        // só a própria classe enxerga
    private int idade;

    // GETTER: devolve o valor (leitura segura)
    public String getNome() {
        return nome;
    }

    // SETTER: altera com regras (escrita controlada)
    public void setNome(String nome) {
        this.nome = nome;
    }
}
```

- Convenção: `getNome()` para ler, `setNome(...)` para alterar.
- De fora, usa-se **métodos**, nunca `pessoa.nome` (nem compila).
- A IDE gera getters/setters automaticamente (Alt+Insert no IntelliJ).

Exemplo: validação dentro do setter

Setters podem rejeitar valores absurdos — é aqui que o encapsulamento brilha:

```
Pessoa.java

public class Pessoa {
    private String nome;
    private int idade;

    public void setIdade(int idade) {
        if (idade < 0) {
            System.out.println("Idade inválida! Usando 0.");
            this.idade = 0;
        } else {
            this.idade = idade;
        }
    }

    public int getIdade() {
        return idade;
    }
}
```

- Agora `pessoa.setIdade(-5)` é corrigido pela própria classe — o estado fica sempre válido.

Exemplo: ContaBancaria encapsulada

Toda movimentação passa por métodos com regras de negócio:

ContaBancaria.java

```
public class ContaBancaria {  
    private double saldo;  
  
    public void depositar(double valor) {  
        if (valor > 0) {  
            saldo = saldo + valor;  
        } else {  
            System.out.println("Depósito inválido.");  
        }  
    }  
  
    public boolean sacar(double valor) {  
        if (valor > 0 && valor <= saldo) {  
            saldo = saldo - valor;  
            return true;           // saque realizado  
        }  
        return false;            // saque recusado  
    }  
  
    public double getSaldo() {  
        return saldo;  
    }  
}
```

- Quem usa a classe **não toca no saldo diretamente** — só via depositar/sacar/getSaldo.

Exemplo: usando a ContaBancaria

O programa principal só conhece os métodos públicos — simples e seguro:

TesteConta.java

```
public class TesteConta {  
    public static void main(String[] args) {  
        ContaBancaria conta = new ContaBancaria();  
        conta.depositar(1000);  
        boolean ok = conta.sacar(300);  
        System.out.println("Saque de 300: " + ok);  
        System.out.println("Saldo: " + conta.getSaldo());  
        ok = conta.sacar(9999);  
        System.out.println("Saque de 9999: " + ok);  
        System.out.println("Saldo: " + conta.getSaldo());  
    }  
}
```

Saída

```
Saque de 300: true  
Saldo: 700.0  
Saque de 9999: false  
Saldo: 700.0
```

- O saque maior que o saldo é **recusado** — a regra vive dentro da classe.

Exercício 1 — Classe Produto encapsulada

- Crie a classe `Produto` com atributos **privados**: `nome`, `preco`, `quantidade`.
- Getters e setters para os três.
- No setter de preço, **rejeite valores negativos** (imprima aviso e mantenha o atual).
- Método `void adicionarEstoque(int qtd)` e `boolean vender(int qtd)` (retorna false se faltar estoque).
- Teste no main: vender mais do que existe deve ser recusado.

Dica: Comece pelos setters simples; depois adicione a validação no `setPreco`.

Resolução — Exercício 1 (Classe Produto)

● ● ● Produto.java

```
public class Produto {  
    private String nome;  
    private double preco;  
    private int quantidade;  
  
    public String getNome() { return nome; }  
    public void setNome(String nome) { this.nome = nome; }  
  
    public double getPreco() { return preco; }  
    public void setPreco(double preco) {  
        if (preco < 0) {  
            System.out.println("Preço inválido! Valor mantido.");  
        } else {  
            this.preco = preco;  
        }  
    }  
  
    public int getQuantidade() { return quantidade; }  
  
    public void adicionarEstoque(int qtd) {  
        quantidade = quantidade + qtd;  
    }  
  
    public boolean vender(int qtd) {  
        if (qtd > 0 && qtd <= quantidade) {  
            quantidade = quantidade - qtd;  
            return true;  
        }  
        return false;  
    }  
}
```

- Versão compacta dos getters/setters em 1 linha — comum em código profissional.

Construtores: por que eles existem?

- Hoje criamos objeto e preenchemos em **3 passos**:
 - `Produto p = new Produto(); → p.setNome(...) → p.setPreco(...)`
- É trabalhoso... e dá para **esquecer** de preencher algo.
- **Construtor** = método especial que roda **na hora do `new`**.
- Ele **inicializa o objeto** com valores — em um único passo:
 - `Produto p = new Produto("Caneta", 2.50, 100);`
- O nome do construtor é **sempre o nome da classe**.
- Construtores **não têm retorno** (nem `void`).

Construtor simples e construtor com parâmetros

Pessoa.java

```
public class Pessoa {  
    private String nome;  
    private int idade;  
  
    // Construtor SEM parâmetros (padrão)  
    public Pessoa() {  
        nome = "Sem nome";  
        idade = 0;  
    }  
  
    // Construtor COM parâmetros – mais útil!  
    public Pessoa(String nome, int idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
}
```

Saída

```
p1: Sem nome / 0  
p2: Ana / 22
```

- `new Pessoa()` → usa o 1º construtor; `new Pessoa("Ana", 22)` → usa o 2º.
- Se você **não criar nenhum**, o Java cria um construtor padrão vazio.
- Se criar um com parâmetros e quiser o vazio, **precisa escrevê-lo**.

Sobrecarga de construtores

● ● ● Livro.java

```
public class Livro {  
    private String titulo;  
    private int paginas;  
  
    public Livro() {  
        this("Sem título", 0);    // delega ao construtor abaixo  
    }  
  
    public Livro(String titulo) {  
        this(titulo, 0);  
    }  
  
    public Livro(String titulo, int paginas) {  
        this.titulo = titulo;  
        this.paginas = paginas;  
    }  
}
```

Saída

```
1) Sem título / 0  
2) Dom Casmurro / 0  
3) Dom Casmurro / 256
```

- **Sobrecarga** = vários construtores com o **mesmo nome**, mas assinaturas diferentes.
- `this(...)` chama outro construtor da mesma classe — evita repetição.

Regras da sobrecarga

- **O que pode variar:** o número de parâmetros e/ou os tipos deles.
- Exemplos válidos:
 - `Livro()` · `Livro(String titulo)` · `Livro(String titulo, int paginas)`
 - `somar(int a, int b)` e `somar(double a, double b)` (tipos diferentes)
- **O que NÃO vale:** mudar apenas o tipo de retorno.
 - `int somar(...)` e `double somar(...)` com mesmos parâmetros → erro.
- O Java escolhe o método **pelo que você passa na chamada**.
- Sobrecarga deixa a classe **flexível**: várias formas de usar.

Sobrecarga de métodos

Calculadora.java

```
public class Calculadora {  
    public int somar(int a, int b) {  
        return a + b;  
    }  
  
    public int somar(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    public double somar(double a, double b) {  
        return a + b;  
    }  
}
```

Saída

```
somar(2, 3)      = 5  
somar(2, 3, 4)   = 9  
somar(1.5, 2.5)  = 4.0
```

- Três métodos com o mesmo nome `somar`, assinaturas diferentes.
- O compilador decide qual usar pelo **tipo/quantidade de argumentos**.

Exemplo: classe Aluno completa

Construtores, encapsulamento, sobrecarga e lógica de negócio juntos:

```
Aluno.java

public class Aluno {
    private String nome;
    private double nota1, nota2;

    public Aluno() { }

    public Aluno(String nome, double nota1, double nota2) {
        this.nome = nome;
        this.nota1 = nota1;
        this.nota2 = nota2;
    }

    public double calcularMedia() {
        return (nota1 + nota2) / 2;
    }

    public boolean foiAprovado() {
        return calcularMedia() >= 6;
    }

    public void exibir() {
        System.out.println(nome + " - média " + calcularMedia()
            + (foiAprovado() ? " (aprovado)" : " (reprovado)"));
    }
}
```

- Um método pode chamar outro método da própria classe (`calcularMedia()`).

Exercício 2 — Classe Funcionário

- Crie a classe `Funcionario` com `private String nome` e `private double salario`.
- Dois construtores: vazio e `(String nome, double salario)`.
- Método `void aumentarSalario(double porcentagem)` (ex.: 10 = 10%).
- Método `double calcularSalarioAnual()` (salário × 13 — inclui 13º).
- No main, crie 2 funcionários de formas diferentes e exiba os salários anuais.

Dica: Porcentagem: `salario = salario + salario * (porcentagem / 100);`

Resolução — Exercício 2 (Classe Funcionário)

Funcionario.java

```
public class Funcionario {  
    private String nome;  
    private double salario;  
  
    public Funcionario() { }  
  
    public Funcionario(String nome, double salario) {  
        this.nome = nome;  
        this.salario = salario;  
    }  
  
    public String getNome() { return nome; }  
    public void setNome(String nome) { this.nome = nome; }  
  
    public void aumentarSalario(double porcentagem) {  
        salario = salario + salario * (porcentagem / 100);  
    }  
  
    public double calcularSalarioAnual() {  
        return salario * 13;  
    }  
}
```

Resolução — Exercício 2 (Classe Funcionário — programa principal)

TesteFuncionario.java

```
public class TesteFuncionario {  
    public static void main(String[] args) {  
        Funcionario ana = new Funcionario("Ana", 3000);  
        Funcionario beto = new Funcionario();  
        beto.setNome("Beto");           // se houver setter...  
  
        ana.aumentarSalario(10);  
        System.out.println(ana.getNome() + ": anual R$ " + ana.calcularSalarioAnual());  
        System.out.println("Beto: anual R$ " + beto.calcularSalarioAnual());  
    }  
}
```

Saída

Ana: anual R\$ 42900.0
Beto: anual R\$ 0.0

- Funcionario agora tem getNome/setNome (veja a resolução).

Atributo de classe: o modificador static

- Até agora, **cada objeto** tem sua própria cópia dos atributos.
- Às vezes o valor deve ser **compartilhado por toda a classe**:
 - contador de quantos objetos foram criados,
 - configuração global (taxa, versão do sistema).
- `static` = o atributo **pertence à classe**, não ao objeto.
- Acessamos: `Classe.atributo` — sem precisar de objeto.
- Existe **uma única cópia** na memória, compartilhada.

Exemplo: contando objetos criados

Cada vez que um Usuario é criado, o contador estático aumenta:

Usuario.java

```
public class Usuario {  
    private static int totalUsuarios = 0;    // da CLASSE  
    private String nome;  
  
    public Usuario(String nome) {  
        this.nome = nome;  
        totalUsuarios++;                    // incrementa o compartilhado  
    }  
  
    public static int getTotalUsuarios() {  
        return totalUsuarios;  
    }  
}
```

Saída

```
Criado: Ana  
Criado: Beto  
Total de usuários: 2
```

- No main: `Usuario u1 = new Usuario("Ana"); ... System.out.println(Usuario.getTotalUsuarios());`

Constantes: final + static

Configuracoes.java

```
public class Configuracoes {  
    // constante de classe: valor fixo, maiúsculo, acessível sem objeto  
    public static final double TAXA_JUROS = 0.02;  
    public static final int MAX_TENTATIVAS = 3;  
    public static final String NOME_SISTEMA = "Sistema Escolar";  
}
```

Saída

Bem-vindo ao Sistema Escolar
Tentativas permitidas: 3

- `final` → o valor **não pode mudar** depois de atribuído.
- `static final` → constante da classe (padrão: UPPER_SNAKE_CASE).
- Uso: `Configuracoes.TAXA_JUROS` — direto, sem objeto.

Métodos estáticos: chamados sem objeto

Util.java

```
public class Util {  
    // método estático: não precisa de objeto para chamar  
    public static double converterKmParaMilhas(double km) {  
        return km * 0.621371;  
    }  
  
    public static boolean ehPar(int numero) {  
        return numero % 2 == 0;  
    }  
}
```

Saída

```
10 km = 6.21371 milhas  
42 é par? true
```

- Chamada: `Util.ehPar(42)` — sem `new`.
- `Math.PI`, `Math.sqrt(...)` são estáticos — você já usava!
- `main` é estático: roda sem precisar criar a própria classe.

Objetos como parâmetros (e retornos)

ContaBancaria.java

```
public class ContaBancaria {  
    private String titular;  
    private double saldo;  
  
    // recebe OUTRO OBJETO como parâmetro  
    public boolean transferir(ContaBancaria destino, double valor) {  
        if (sacar(valor)) {           // saca da própria conta  
            destino.depositar(valor);  // deposita na outra  
            return true;  
        }  
        return false;  
    }  
  
    public void depositar(double valor) { saldo += valor; }  
    public boolean sacar(double valor) {  
        if (valor > 0 && valor <= saldo) { saldo -= valor; return true; }  
        return false;  
    }  
    public double getSaldo() { return saldo; }  
}
```

Saída

Ana: R\$ 700.0 | Beto: R\$ 1300.0

- No main: `ana.transferir(beto, 300);` — objetos interagem entre si.

== versus equals: cuidado com objetos!

Comparacao.java

```
public class Comparacao {  
    public static void main(String[] args) {  
        String a = new String("Ana");  
        String b = new String("Ana");  
  
        System.out.println(a == b);           // false! compara referências  
        System.out.println(a.equals(b));       // true! compara conteúdo  
  
        int x = 5, y = 5;  
        System.out.println(x == y);           // true (primitivos: valor)  
    }  
}
```

Saída

```
false  
true  
true
```

- `==` compara **endereço de memória** em objetos.
- `.equals(...)` compara o **conteúdo/valor lógico**.
- Para primitivos (`int`, `double...`) `==` compara valores — ok.

toString: como o objeto 'se descreve'

Pessoa.java

```
public class Pessoa {  
    private String nome;  
    private int idade;  
  
    public Pessoa(String nome, int idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
  
    @Override  
    public String toString() {  
        return "Pessoa{nome='" + nome + "', idade=" + idade + "}";  
    }  
}
```

Saída

Pessoa{nome='Ana', idade=22}

- `toString()` define o **texto padrão** do objeto.
- Basta `System.out.println(pessoa);` — o Java chama o `toString`.
- Sem ele, você vê algo como `Pessoa@3f200a`.

Exemplo completo: estoque com encapsulamento

Cadastro de produtos com preço, estoque e impressão formatada:

TesteEstoque.java

```
public class TesteEstoque {  
    public static void main(String[] args) {  
        Produto caneta = new Produto();  
        caneta.setNome("Caneta azul");  
        caneta.setPreco(2.50);  
        caneta.adicionarEstoque(100);  
  
        System.out.println(caneta.getNome() + " custa R$ " + caneta.getPreco());  
        boolean vendeu = caneta.vender(50);  
        System.out.println("Vendeu 50? " + vendeu + " – sobraram " + caneta.getQuantidade() + "?");  
    }  
}
```

Saída

```
Caneta azul custa R$ 2.5  
Vendeu 50? true – sobraram 50?
```

- Usa a classe Produto resolvida no Exercício 1 (com getters).

Exercício 3 — Integrador: ContaCorrente com transferência

- Crie `ContaCorrente` com `private String titular` e `private double saldo`.
- Construtor `ContaCorrente(String titular, double saldoInicial)`.
- Métodos: `depositar(double)`, `sacar(double)` (sem deixar negativo), `getSaldo()`.
- Método `boolean transferir(ContaCorrente destino, double valor)`.
- No main, crie 2 contas, transfira e mostre os saldos finais.

Dica: `transferir` deve usar os métodos `sacar` e `depositar` do próprio objeto e do destino.

Resolução — Exercício 3 (ContaCorrente)

ContaCorrente.java

```
public class ContaCorrente {
    private String titular;
    private double saldo;
    public ContaCorrente(String titular, double saldoInicial) {
        this.titular = titular;
        this.saldo = saldoInicial;
    }
    public void depositar(double valor) {
        if (valor > 0) saldo += valor;
    }
    public boolean sacar(double valor) {
        if (valor > 0 && valor <= saldo) {
            saldo -= valor;
            return true;
        }
        return false;
    }
    public boolean transferir(ContaCorrente destino, double valor) {
        if (sacar(valor)) {
            destino.depositar(valor);
            return true;
        }
        return false;
    }
    public double getSaldo() { return saldo; }
    public String getTitular() { return titular; }
}
```

Resolução — Exercício 3 (ContaCorrente — teste)

● ● ● TesteTransferencia.java

```
public class TesteTransferencia {  
    public static void main(String[] args) {  
        ContaCorrente ana = new ContaCorrente("Ana", 1000);  
        ContaCorrente beto = new ContaCorrente("Beto", 500);  
  
        boolean ok = ana.transferir(beto, 350);  
        System.out.println("Transferência: " + ok);  
        System.out.println(ana.getTitular() + ": R$ " + ana.getSaldo());  
        System.out.println(beto.getTitular() + ": R$ " + beto.getSaldo());  
    }  
}
```

Saída

```
Transferência: true  
Ana: R$ 650.0  
Beto: R$ 850.0
```

Erros comuns do Módulo 2

● ● ● Erros.java

```
// ERRO 1: atributo private acessado de fora
Pessoa p = new Pessoa();
p.nome = "Ana";           // não compila! (nome é private)
p.setNome("Ana");         // certo

// ERRO 2: esquecer return em método que devolve valor
double calcular() { return 0; } // todo caminho precisa de return

// ERRO 3: variável local com mesmo nome escondendo o atributo
void testar(String nome) {
    nome = nome;           // não faz nada! faltou this.nome
}
```

- Private é do objeto: só métodos **da própria classe** acessam.
- Se o método 'devolve' valor e não tem return em algum caminho → erro.
- `nome = nome` é inútil — use `this.nome = nome`.

Boas práticas de classes e métodos

- **Atributos privados** por padrão; expor apenas o necessário.
- **Validar** nos setters e nos métodos que mudam o estado.
- **Construtores** para garantir objetos sempre válidos.
- Métodos pequenos e com **uma responsabilidade** só.
- Nomes de métodos = **verbos**: `calcularMedia`, `vender`, `exibir`.
- Use `this` para deixar claro que se refere ao atributo.
- `toString` para facilitar a impressão e o debug.

Resumo do Módulo 2

- **Encapsulamento:** `private` nos atributos + métodos públicos controlam o acesso.
- **Getters/setters** leem e alteram com validação.
- **Construtor** inicializa o objeto no `new`; **sobrecarga** = várias assinaturas.
- `this()` chama outro construtor; `this.atributo` desambigua.
- ``static`` pertence à classe (contador, constantes, métodos utilitários).
- Objetos podem **receber outros objetos** e devolvê-los (transferência).
- Compare conteúdo com ``equals``, não com `==`; use ``toString``.

Checkpoint rápido — Módulo 2

1. Qual modificador protege um atributo de acesso externo?

- (A) public
- (B) private
- (C) final

2. Construtor é:

- (A) Um método sem retorno com o nome da classe
- (B) Um tipo de variável
- (C) Um loop

3. O que significa sobrecarga?

- (A) Herdar de outra classe
- (B) Vários métodos com o mesmo nome e assinaturas diferentes
- (C) Métodos com retorno

Gabarito: 1→B, 2→A, 3→B

Módulo 3 · 6 horas

Relacionamentos entre classes

Objetos que trabalham juntos: composição, listas e coleções

-
- ✓ Entender associação, agregação e composição
 - ✓ Armazenar objetos em arrays e ArrayList
 - ✓ Percorrer listas com for-each
 - ✓ Dominar a classe String
 - ✓ Usar enum para valores fixos
 - ✓ Construir um mini-sistema escolar

Agenda do módulo

1. Por que objetos se relacionam

- Sistemas = rede de objetos trabalhando juntos

2. Tipos de relacionamento

- Associação, agregação, composição

3. Guardando objetos

- Array de objetos, ArrayList, for-each

4. String e enum

- Métodos essenciais de String, enums

5. Exercícios

- Turma de alunos, agenda de contatos, pedidos

6. Resumo + quiz

- Pontos-chave e verificação

Sistemas reais são redes de objetos

- Um sistema escolar tem **Aluno, Professor, Turma, Disciplina...**
- Esses conceitos **se relacionam**: uma Turma tem alunos, um Professor ministra Disciplinas.
- Na POO, modelamos isso fazendo classes **se referenciarem** umas às outras.
- Um atributo pode ser de **outra classe**:
 - `private Professor professor;` dentro da classe Turma.
- Um atributo pode ser uma **lista de objetos**:
 - `private ArrayList<Aluno> alunos;` dentro da classe Turma.
- É assim que pequenos blocos viram **sistemas grandes**.

Associação, agregação e composição

Tipo	Ideia	Exemplo	Vida independente?
Associação	objetos se conhecem e se usam	Aluno usa Caderno	Sim, total
Agregação	tem-um fraco	Carro tem Motor	Sim — o motor existe sem o carro
Composição	tem-um forte, parte-todo	Casa tem Comodos	Não — sem casa, sem comodos

Exemplo: associação — objetos que se usam

Um Aluno conhece sua Cidade; a Cidade não depende do Aluno:

Aluno.java

```
public class Aluno {  
    private String nome;  
    private Cidade cidade;    // atributo de OUTRA classe!  
  
    public Aluno(String nome, Cidade cidade) {  
        this.nome = nome;  
        this.cidade = cidade;  
    }  
  
    public void exibir() {  
        System.out.println(nome + " mora em " + cidade.getNome());  
    }  
}
```

- Para criar: `new Aluno("Ana", new Cidade("Londrina"))`.

Agregação: Carro tem Motor

Carro.java

```
class Motor {                      // auxiliar no mesmo arquivo
    private int potencia;           // em cavalos
    public Motor(int potencia) { this.potencia = potencia; }
    public int getPotencia() { return potencia; }
}

public class Carro {
    private String modelo;
    private Motor motor;            // agregação: Carro TEM Motor

    public Carro(String modelo, Motor motor) {
        this.modelo = modelo;
        this.motor = motor;         // motor veio pronto de fora
    }

    public void exibir() {
        System.out.println(modelo + " com motor de "
            + motor.getPotencia() + " cv");
    }
}
```

Saída

Fusca com motor de 65 cv

- O Motor pode ser criado antes e passado: `new Carro("Fusca", new Motor(65)).`

Composição: Casa tem Comodos

Casa.java

```
class Comodo {                // auxiliar no mesmo arquivo
    private String nome;
    public Comodo(String nome) { this.nome = nome; }
    public String getNome() { return nome; }
}

public class Casa {
    private Comodo[] comodos;    // composição forte

    public Casa(String[] nomesDosComodos) {
        comodos = new Comodo[nomesDosComodos.length];
        for (int i = 0; i < nomesDosComodos.length; i++) {
            comodos[i] = new Comodo(nomesDosComodos[i]);
        }
    }

    public void listar() {
        for (Comodo c : comodos) {
            System.out.println("Comodo: " + c.getNome());
        }
    }
}
```

Saída

```
Comodo: Sala
Comodo: Quarto
Comodo: Cozinha
```

- Os comodos nascem **dentro** da Casa — sem casa, eles não existem.

Array de objetos: guardando vários

● ● ● TesteBiblioteca.java

```
class Livro {                      // auxiliar no mesmo arquivo
    private String titulo;
    public Livro(String titulo) { this.titulo = titulo; }
    public String getTitulo() { return titulo; }
}

public class TesteBiblioteca {
    public static void main(String[] args) {
        Livro[] acervo = new Livro[3]; // vetor de 3 referências
        acervo[0] = new Livro("Dom Casmurro");
        acervo[1] = new Livro("Memórias Póstumas");
        acervo[2] = new Livro("Quincas Borba");
        for (int i = 0; i < acervo.length; i++) {
            System.out.println(acervo[i].getTitulo());
        }
    }
}
```

Saída

Dom Casmurro
Memórias Póstumas
Quincas Borba

- `new Livro[3]` cria 3 'gavetas' para objetos — não os objetos ainda.
- Depois preenchamos cada posição com `new Livro(...)`.
- Problema: tamanho **fixo**. Para crescer, vem o ArrayList!

ArrayList: a lista que cresce sozinha

● ● ● ListaDinamica.java

```
import java.util.ArrayList;

public class ListaDinamica {
    public static void main(String[] args) {
        ArrayList<String> nomes = new ArrayList<>();

        nomes.add("Ana");
        nomes.add("Beto");
        nomes.add("Carla");

        System.out.println("Tamanho: " + nomes.size());
        System.out.println("Primeiro: " + nomes.get(0));
        nomes.remove(1); // remove o Beto
        System.out.println("Contém Carla? " + nomes.contains("Carla"));
        System.out.println("Lista: " + nomes);
    }
}
```

Saída

```
Tamanho: 3
Primeiro: Ana
Contém Carla? true
Lista: [Ana, Carla]
```

- `add`, `get(i)`, `size()`, `remove(i)`, `contains(...)` — os principais.
- `<String>` diz o tipo: a lista só aceita String (é genérica).
- Precisa de `import java.util.ArrayList;` no topo.

Percorrendo com for-each

ForEach.java

```
import java.util.ArrayList;

public class ForEach {
    public static void main(String[] args) {
        ArrayList<String> frutas = new ArrayList<>();
        frutas.add("Maçã");
        frutas.add("Banana");
        frutas.add("Uva");

        // for-each: para cada fruta na lista...
        for (String fruta : frutas) {
            System.out.println("Gosto de " + fruta);
        }
    }
}
```

Saída

```
Gosto de Maçã
Gosto de Banana
Gosto de Uva
```

- Sintaxe: `for (Tipo variavel : colecao).`
- A variável recebe **cada elemento** na ordem.
- Mais simples que o `for` com índice quando não precisamos do índice.

Exemplo: Biblioteca com ArrayList de livros

A biblioteca guarda quantos livros quiser e lista todos:

● ● ● Biblioteca.java

```
import java.util.ArrayList;

public class Biblioteca {
    private ArrayList<Livro> acervo = new ArrayList<>();

    public void adicionar(Livro livro) {
        acervo.add(livro);
    }

    public void listar() {
        for (Livro livro : acervo) {
            System.out.println("- " + livro.getTitulo());
        }
    }
}
```

Saída

```
- Dom Casmurro
- O Cortiço
- Iracema
```

- No main: criar biblioteca, `biblioteca.adicionar(new Livro(...))` várias vezes, e `biblioteca.listar()`.

A classe String em detalhe

● ● ● Strings.java

```
public class Strings {  
    public static void main(String[] args) {  
        String nome = "João da Silva";  
        System.out.println("Tamanho: " + nome.length());  
        System.out.println("Maiúsculas: " + nome.toUpperCase());  
        System.out.println("Começa com Jo? " + nome.startsWith("Jo"));  
        System.out.println("Contém Silva? " + nome.contains("Silva"));  
        String sub = nome.substring(5, 9); // da posição 5 até a 8  
        System.out.println("Pedacinho: " + sub);  
        System.out.println("Trocou: " + nome.replace("João", "José"));  
    }  
}
```

Saída

```
Tamanho: 13  
Maiúsculas: JOÃO DA SILVA  
Começa com Jo? true  
Contém Silva? true  
Pedacinho: da S  
Trocou: José da Silva
```

- String é **imutável**: cada 'mudança' gera uma nova String.
- Compare com `equals` (nunca `==`), e ignore maiúsculas com `equalsIgnoreCase`.

enum: valores fixos com significado

● ● ● DiaDaSemana.java

```
public enum DiaDaSemana {  
    SEGUNDA, TERCA, QUARTA, QUINTA, SEXTA, SABADO, DOMINGO  
}
```

Saída

Hoje é: SEXTA
É fim de semana? false

- `enum` define um conjunto **fechado** de valores.
- Perfeito para dias, status de pedido, categorias, tamanhos.
- Uso: `DiaDaSemana hoje = DiaDaSemana.SEXTA;`
- Código fica **mais seguro**: valores inválidos nem compilam.

enum na prática: status de pedido

● ● ● Pedido.java

```
enum StatusPedido {           // auxiliar no mesmo arquivo
    PENDENTE, PAGO, ENVIADO, ENTREGUE, CANCELADO
}

public class Pedido {
    private int numero;
    private StatusPedido status;

    public Pedido(int numero, StatusPedido status) {
        this.numero = numero;
        this.status = status;
    }

    public void avancar() {
        switch (status) {
            case PENDENTE: status = StatusPedido.PAGO; break;
            case PAGO:      status = StatusPedido.ENVIADO; break;
            case ENVIADO:   status = StatusPedido.ENTREGUE; break;
            default:        System.out.println("Não avança: " + status);
        }
    }
}
```

Saída

Pedido 1: PENDENTE
Pedido 1: ENVIADO

- No main: `new Pedido(1, StatusPedido.PENDENTE)`, chamar `avancar()` 2x e exibir.

Exercício 1 — Sistema: Turma de alunos

- Classe ``Aluno`` (privados): `nome`, `nota`; construtor + getters.
- Classe ``Turma``: `private ArrayList<Aluno> alunos;`
- Métodos: `adicionar(Aluno)`, `listar()` (mostra nome e nota),
- `calcularMediaDaTurma()` — some as notas e divida pelo tamanho.
- No main: crie 3 alunos, adicione à turma, liste e mostre a média.

Dica: Comece com `new ArrayList<>()`; para a média, percorra com for-each.

Resolução — Exercício 1 (Turma — classe Aluno)

Aluno.java

```
public class Aluno {  
    private String nome;  
    private double nota;  
  
    public Aluno(String nome, double nota) {  
        this.nome = nome;  
        this.nota = nota;  
    }  
  
    public String getNome() { return nome; }  
    public double getNota() { return nota; }  
}
```

Resolução — Exercício 1 (Turma — classe Turma)

Turma.java

```
import java.util.ArrayList;

public class Turma {
    private ArrayList<Aluno> alunos = new ArrayList<>();

    public void adicionar(Aluno aluno) {
        alunos.add(aluno);
    }

    public void listar() {
        for (Aluno a : alunos) {
            System.out.println(a.getNome() + " - nota " + a.getNota());
        }
    }

    public double calcularMediaDaTurma() {
        double soma = 0;
        for (Aluno a : alunos) {
            soma += a.getNota();
        }
        return alunos.isEmpty() ? 0 : soma / alunos.size();
    }
}
```

Resolução — Exercício 1 (Turma — programa principal)

TesteTurma.java

```
public class TesteTurma {  
    public static void main(String[] args) {  
        Turma turma = new Turma();  
        turma.adicionar(new Aluno("Ana", 8.5));  
        turma.adicionar(new Aluno("Beto", 7.0));  
        turma.adicionar(new Aluno("Carla", 9.5));  
  
        turma.listar();  
        System.out.println("Média da turma: " + turma.calcularMediaDaTurma());  
    }  
}
```

Saída

```
Ana — nota 8.5  
Beto — nota 7.0  
Carla — nota 9.5  
Média da turma: 8.333333333333334
```

- `alunos.isEmpty()` evita dividir por zero quando a turma está vazia.

Exercício 2 — Agenda de contatos

- Classe `Contato` (privados): `nome`, `telefone`; construtor + getters.
- Classe `Agenda`: `ArrayList<Contato>` com:
 - `adicionar(Contato)`, `listar()`, `buscar(String nome)` (contém o trecho, ignore maiúsculas).
- No main: adicione 3 contatos, liste, busque por um nome.

Dica: Busca: percorra com for-each e use `nome.contains(...)` ou `equalsIgnoreCase`.

Resolução — Exercício 2 (Agenda — classes Contato e Agenda)

● ● ● Agenda.java

```
import java.util.ArrayList;
class Contato {                // sem public: usada no mesmo pacote
    private String nome;
    private String telefone;
    public Contato(String nome, String telefone) {
        this.nome = nome;
        this.telefone = telefone;
    }
    public String getNome() { return nome; }
    public String getTelefone() { return telefone; }
}
public class Agenda {
    private ArrayList<Contato> contatos = new ArrayList<>();
    public void adicionar(Contato c) { contatos.add(c); }
    public void listar() {
        for (Contato c : contatos)
            System.out.println(c.getNome() + " — " + c.getTelefone());
    }
    public void buscar(String trecho) {
        for (Contato c : contatos) {
            if (c.getNome().toLowerCase().contains(trecho.toLowerCase())) {
                System.out.println("Achei: " + c.getNome() + " — " + c.getTelefone());
            }
        }
    }
}
```

Exercício 3 — Mini sistema de pedidos

- Classe `ItemPedido`: `descricao`, `quantidade`, `precoUnitario` (construtor + getters).
- Método `double getTotal()` → `quantidade × precoUnitario`.
- Classe `Pedido`: `ArrayList<ItemPedido>` + `void adicionarItem(ItemPedido)`.
- Método `double getValorTotal()` → soma dos totais dos itens.
- Desafio: `exibir()` mostra cada item e o total geral no final.

Dica: Um método da classe `ItemPedido` calcula o total de UM item; o `Pedido` soma os itens.

Resolução — Exercício 3 (Pedidos — ItemPedido)

ItemPedido.java

```
public class ItemPedido {  
    private String descricao;  
    private int quantidade;  
    private double precoUnitario;  
  
    public ItemPedido(String descricao, int quantidade, double precoUnitario) {  
        this.descricao = descricao;  
        this.quantidade = quantidade;  
        this.precoUnitario = precoUnitario;  
    }  
  
    public double getTotal() {  
        return quantidade * precoUnitario;  
    }  
  
    public String getDescricao() { return descricao; }  
}
```

Resolução — Exercício 3 (Pedidos — Pedido e teste)

TestePedido.java

```
import java.util.ArrayList;

class Pedido {
    // auxiliar no mesmo arquivo
    private ArrayList<ItemPedido> itens = new ArrayList<>();

    public void adicionarItem(ItemPedido item) { itens.add(item); }

    public double getValorTotal() {
        double total = 0;
        for (ItemPedido item : itens) {
            total += item.getTotal();
        }
        return total;
    }
}

public class TestePedido {
    public static void main(String[] args) {
        Pedido pedido = new Pedido();
        pedido.adicionarItem(new ItemPedido("Caneta", 10, 2.50));
        pedido.adicionarItem(new ItemPedido("Caderno", 2, 18.00));
        System.out.println("Total do pedido: R$ " + pedido.getValorTotal());
    }
}
```

Saída

Total do pedido: R\$ 61.0

Diagramas de classes: o 'mapa' do sistema

- Antes de codar sistemas grandes, desenhamos o **diagrama de classes** (UML).
- Cada classe é um retângulo com 3 partes:
 - nome da classe · atributos · métodos.
- As **setas** mostram os relacionamentos:
 - linha simples → associação;
 - losango vazio (◊) → agregação;
 - losango cheio (◆) → composição.
- Exemplo mental: **Turma** ◆— **Aluno** (composição).
- No Módulo 5 desenharemos o diagrama do **projeto final**.

Resumo do Módulo 3

- Objetos **se relacionam**: associação (usa), agregação (tem), composição (parte-todo).
- **Array de objetos**: tamanho fixo; **ArrayList**: cresce sozinho (`add`, `get`, `size`).
- **for-each** percorre coleções de forma simples e segura.
- **String**: `length`, `toUpperCase`, `contains`, `startsWith`, `substring`, `replace`.
- **enum** garante valores fixos e legíveis (`StatusPedido.PAGO`).
- Sistemas = classes que se compõem: Turma com Alunos, Pedido com Itens.

Checkpoint rápido — Módulo 3

1. Qual estrutura cresce dinamicamente?

- (A) Array fixo
- (B) ArrayList
- (C) int

2. Em composição, as partes:

- (A) Vivem sem o todo
- (B) Nascem e morrem com o todo
- (C) Não se relacionam

3. Para comparar conteúdo de duas Strings, usamos:

- (A) ==
- (B) .equals()
- (C) .length()

Gabarito: 1→B, 2→B, 3→B

Módulo 4 · 8 horas

Herança e Polimorfismo

Reutilize código e escreva sistemas flexíveis

-
- ✓ Reutilizar código com herança (extends)
 - ✓ Reescrever métodos (override) com @Override
 - ✓ Tratar objetos de várias classes como um tipo só (polimorfismo)
 - ✓ Usar classes e métodos abstratos
 - ✓ Definir contratos com interfaces
 - ✓ Construir uma folha de pagamento polimórfica

Agenda do módulo

1. Motivação

- Código repetido entre Funcionario, Gerente, Diretor

2. Herança

- extends, super, protected, override

3. Polimorfismo

- upcasting, arrays polimórficos, instanceof

4. Classes abstratas

- Métodos abstratos, molde incompleto

5. Interfaces

- Contratos de comportamento, interface × abstrata

6. Exercícios + resumo

- Animais, veículos, folha de pagamento + quiz

O problema: código repetido

- Num sistema, `Funcionario`, `Gerente` e `Diretor` têm muito em comum:
 - todos têm `nome` e `salario`; todos têm `exibir()` e `calcularPagamento()`.
- Sem herança, **copiaríamos e colaríamos** o código 3 vezes.
- Consequências ruins:
 - corrigir um bug → corrigir em 3 lugares;
 - adicionar campo (email) → editar 3 classes;
 - o código cresce e fica difícil de manter.
- **Herança** resolve: criamos a classe base e as outras **herdam** dela.

Herança: o que é

- **Herança** = uma classe **herda** atributos e métodos de outra.
- Vocabulário:
 - **superclasse** (classe mãe / base): `Funcionario`,
 - **subclasse** (classe filha / derivada): `Gerente`.
- `Gerente` **é um** `Funcionario` (relação 'é-um').
- Palavra-chave: `extends`.
- A subclasse pode:
 - **usar** o que herdou (atributos/métodos públicos e protected),
 - **adicionar** atributos e métodos próprios,
 - **reescrever** (override) métodos herdados.

Aplicando: Funcionario e Gerente

Funcionario.java

```
public class Funcionario {  
    protected String nome;           // protected: subclasses enxergam  
    protected double salario;  
  
    public Funcionario(String nome, double salario) {  
        this.nome = nome;  
        this.salario = salario;  
    }  
  
    public void exibir() {  
        System.out.println(nome + " recebe R$ " + salario);  
    }  
}
```

Aplicando: Gerente herda de Funcionario

```
● ● ● Gerente.java

public class Gerente extends Funcionario {    // herda!
    private double bonus;                    // atributo NOVO

    public Gerente(String nome, double salario, double bonus) {
        super(nome, salario);                // chama o construtor da superclasse
        this.bonus = bonus;
    }

    public double getBonus() { return bonus; }
    @Override                                // reescrevendo o comportamento
    public void exibir() {
        System.out.println("Gerente " + nome + " recebe R$ "
            + (salario + bonus));
    }
}
```

- `extends` → Gerente ganha `nome`, `salario` e `exibir()` da base.
- `super(...)` chama o construtor da superclasse (sempre na 1ª linha).
- `@Override` avisa: 'estou reescrevendo um método herdado'.

Exemplo: herança na prática

Gerente usa o que herdou e ainda adiciona o próprio comportamento:

TesteEmpresa.java

```
public class TesteEmpresa {  
    public static void main(String[] args) {  
        Funcionario ana = new Funcionario("Ana", 3000);  
        Gerente carlos = new Gerente("Carlos", 6000, 2000);  
  
        ana.exibir();  
        carlos.exibir();    // usa a versão do Gerente  
    }  
}
```

Saída

```
Ana recebe R$ 3000.0  
Gerente Carlos recebe R$ 8000.0
```

- `protected` permite a subclasse acessar `nome` e `salario` diretamente.

Polimorfismo: 'muitas formas'

- **Polimorfismo** = tratar objetos de classes diferentes **como se fossem do mesmo tipo**.
- Como `Gerente` É UM `Funcionario`, podemos guardá-lo numa variável de `Funcionario`:
 - `Funcionario f = new Gerente(...);` → **upcasting** (sobe na hierarquia).
- Ao chamar `f.exibir()`, o Java executa o método **da classe real do objeto**.
- Resultado: um `ArrayList<Funcionario>` pode conter funcionários E gerentes.
- O código de quem usa **não precisa saber** qual é o tipo exato.

E mais um cargo: Diretor

● ● ● Diretor.java

```
public class Diretor extends Funcionario {  
    private double bonusExtra;  
    public Diretor(String nome, double salario, double bonusExtra) {  
        super(nome, salario);  
        this.bonusExtra = bonusExtra;  
    }  
    @Override  
    public void exibir() {  
        System.out.println("Diretor " + nome + " recebe R$ "  
            + (salario + bonusExtra));  
    }  
}
```

- Diretor também é Funcionario: herda nome/salario e reescreve exibir().

Exemplo: folha de pagamento polimórfica

Um único loop processa funcionários, gerentes e diretores:

```
FolhaPagamento.java

import java.util.ArrayList;
public class FolhaPagamento {
    public static void main(String[] args) {
        ArrayList<Funcionario> equipe = new ArrayList<>();
        equipe.add(new Funcionario("Ana", 3000));
        equipe.add(new Gerente("Carlos", 6000, 2000));
        equipe.add(new Diretor("Sofia", 12000, 5000));

        for (Funcionario f : equipe) {
            f.exibir();    // cada um mostra o SEU valor
        }
    }
}
```

Saída

```
Ana recebe R$ 3000.0
Gerente Carlos recebe R$ 8000.0
Diretor Sofia recebe R$ 17000.0
```

- Diretor também `extends Funcionario`, reescrevendo `exibir()`. O loop não muda para novos cargos!

instanceof e downcasting

TesteEquipe.java

```
public class TesteEquipe {  
    public static void main(String[] args) {  
        Funcionario f = new Gerente("Carlos", 6000, 2000);  
  
        if (f instanceof Gerente) {           // f é um Gerente?  
            Gerente g = (Gerente) f;         // downcasting: desce  
            System.out.println("Bônus: " + g.getBonus());  
        } else {  
            System.out.println("Não é gerente.");  
        }  
    }  
}
```

Saída

Bônus: 2000.0

- `instanceof` verifica o **tipo real** do objeto em tempo de execução.
- Downcasting **só** funciona após verificar — senão: `ClassCastException`.
- Uso comum: acessar métodos que só existem na subclasse.

Exercício 1 — Hierarquia de animais

- Classe base `Animal`: `nome` (protected), construtor, método `void emitirSom()`.
- `Cachorro` `extends Animal`: reescreva `emitirSom()` → "Au au!"
- `Gato` `extends Animal`: reescreva → "Miau!"
- No main, um `ArrayList<Animal>` com os dois; loop chamando `emitirSom()`.

Dica: Todos os animais têm `emitirSom()`, mas cada um faz diferente — polimorfismo puro.

Resolução — Exercício 1 (Animais — classes)

Animais.java

```
class Animal { // arquivo: Animais.java
    protected String nome;
    public Animal(String nome) { this.nome = nome; }
    public void emitirSom() {
        System.out.println(nome + " faz um som genérico.");
    }
}

class Cachorro extends Animal {
    public Cachorro(String nome) { super(nome); }
    @Override
    public void emitirSom() { System.out.println(nome + ": Au au!"); }
}

class Gato extends Animal {
    public Gato(String nome) { super(nome); }
    @Override
    public void emitirSom() { System.out.println(nome + ": Miau!"); }
}
```

Resolução — Exercício 1 (Animais — teste polimórfico)

TesteAnimais.java

```
import java.util.ArrayList;

public class TesteAnimais {
    public static void main(String[] args) {
        ArrayList<Animal> animais = new ArrayList<>();
        animais.add(new Cachorro("Rex"));
        animais.add(new Gato("Mimi"));

        for (Animal a : animais) {
            a.emitirSom();    // polimorfismo: cada um do seu jeito
        }
    }
}
```

Saída

Rex: Au au!
Mimi: Miau!

Classes abstratas: o molde incompleto

- Às vezes a superclasse é **genérica demais** para ter implementação real.
- Exemplo: `Forma` — qual é a área de uma 'forma' genérica? Não existe!
- **Classe abstrata** = não pode ser instanciada (`new Forma()` é proibido).
- Serve como **base** para subclasses completarem.
- **Método abstrato** = só a assinatura, sem corpo; cada subclasse **obrigatoriamente** implementa.
- Sintaxe: `public abstract class Forma` e `public abstract double area();`

Forma abstrata + Circulo e Retangulo

Formas.java

```
abstract class Forma {           // arquivo: Formas.java
    public abstract double area(); // sem corpo: subclasse decide
}

class Circulo extends Forma {
    private double raio;
    public Circulo(double raio) { this.raio = raio; }
    @Override
    public double area() { return Math.PI * raio * raio; }
}

class Retangulo extends Forma {
    private double base, altura;
    public Retangulo(double base, double altura) {
        this.base = base; this.altura = altura;
    }
    @Override
    public double area() { return base * altura; }
}
```

Saída

Círculo: 78.53981633974483
Retângulo: 15.0

- `new Forma()` não compila. Mas `Forma f = new Circulo(5);` compila — polimorfismo!

Interfaces: o contrato de comportamento

- **Interface** define **o que** uma classe faz, sem dizer **como**.
- É um **contrato**: quem implementa, obriga-se a fornecer os métodos.
- Palavra-chave: `implements` — a classe pode implementar **várias** interfaces.
- Exemplo: `Imprimivel` exige `void imprimir()`.
- Qualquer classe pode implementar: `Relatorio`, `Aluno`, `ContaBancaria`...
- Uso típico: mesma ação para tipos diferentes (imprimir, salvar, pagar).

Interface Imprimivel na prática

● ● ● Imprimivel.java

```
interface Imprimivel {           // arquivo: Imprimivel.java
    void imprimir();             // só a assinatura (é public abstract)
}

class Relatorio implements Imprimivel {
    private String titulo;
    public Relatorio(String titulo) { this.titulo = titulo; }
    @Override
    public void imprimir() {
        System.out.println("=== " + titulo + " ===");
    }
}

class Recibo implements Imprimivel {
    private double valor;
    public Recibo(double valor) { this.valor = valor; }
    @Override
    public void imprimir() {
        System.out.println("Recibo no valor de R$ " + valor);
    }
}
```

Saída

```
=== Relatório de Vendas ===
Recibo no valor de R$ 99.9
```

- No main: `ArrayList<Imprimivel>` com os dois, loop `imprimir()`.

Classe abstrata × Interface

CLASSE ABSTRATA

- Pode ter **atributos e métodos concretos**.
- Pode ter **construtores**.
- Representa um **é-um** ('é uma forma').
- Herança **única** (1 só superclasse).
- Ex.: `abstract class Forma`

INTERFACE

- Só assinaturas (até Java 8; depois, defaults).
- **Sem construtores**.
- Representa um **faz/é capaz de** ('é imprimível').
- Uma classe pode implementar **várias**.
- Ex.: `interface Imprimivel`

Exemplo: sistema de pagamento com interface

Meios de pagamento diferentes, uma única chamada: pagar():

```
Pagamento.java

interface Pagamento {           // arquivo: Pagamento.java
    boolean pagar(double valor);
}

class Cartao implements Pagamento {
    public boolean pagar(double valor) {
        System.out.println("Cartão: aprovado R$ " + valor);
        return true;
    }
}

class Pix implements Pagamento {
    public boolean pagar(double valor) {
        System.out.println("Pix: R$ " + valor + " enviado");
        return true;
    }
}

class Boleto implements Pagamento {
    public boolean pagar(double valor) {
        System.out.println("Boleto de R$ " + valor + " gerado");
        return false; // compensa em 1 dia útil
    }
}
```

- A loja só conhece **Pagamento** — adicionar um novo meio não muda o resto do sistema.

Exercício 2 — Veículos com interface

- Interface **Veiculo**: `void acelerar()`, `void frear()`.
- **Carro** implements **Veiculo**: `acelerar` → "Carro acelerando...", `frear` → "Carro freando..."
- **Moto** implements **Veiculo**: mensagens próprias.
- No main: `ArrayList<Veiculo>` com um de cada; loop chamando `acelerar()`.

Dica: A interface garante que TODO veículo sabe acelerar e frear.

Resolução — Exercício 2 (Veículos)

● ● ● Veiculos.java

```
interface Veiculo {                // arquivo: Veiculos.java
    void acelerar();
    void frear();
}

class Carro implements Veiculo {
    @Override public void acelerar() { System.out.println("Carro acelerando..."); }
    @Override public void frear()    { System.out.println("Carro freando..."); }
}

class Moto implements Veiculo {
    @Override public void acelerar() { System.out.println("Moto acelerando..."); }
    @Override public void frear()    { System.out.println("Moto freando..."); }
}
```

Saída

Carro acelerando...
Moto acelerando...

Object: a superclasse de tudo

- **Toda classe** em Java herda de `Object` (direta ou indiretamente).
- É por isso que todo objeto tem:
 - `toString()` → texto representando o objeto,
 - `equals(Object)` → comparação lógica,
 - `hashCode()` → código numérico (usado em coleções).
- Ao reescrever, use `@Override` para manter o contrato.
- Hierarquia: `Gerente` → `Funcionario` → `Object`.

Exercício 3 — Integrador: folha de pagamento com Interface

- Interface `Pagavel`: `double calcularPagamento();`
- `Funcionario` implements `Pagavel`: `pagamento = salário.`
- `Freelancer` implements `Pagavel`: `pagamento = horas × valorHora.`
- No main, `ArrayList<Pagavel>` com ambos; some tudo num loop.
- Desafio: crie uma terceira classe que implementa `Pagavel` sem mudar o loop.

Dica: O loop soma `calcularPagamento()` de cada um — funciona para qualquer nova classe.

Resolução — Exercício 3 (Folha de pagamento com Interface)

Pagavel.java

```
interface Pagavel { // arquivo: Pagavel.java
    double calcularPagamento();
}

class Funcionario implements Pagavel {
    private double salario;
    public Funcionario(double salario) { this.salario = salario; }
    @Override public double calcularPagamento() { return salario; }
}

class Freelancer implements Pagavel {
    private double horas, valorHora;
    public Freelancer(double horas, double valorHora) {
        this.horas = horas; this.valorHora = valorHora;
    }
    @Override public double calcularPagamento() { return horas * valorHora; }
}
```

Resolução — Exercício 3 (Folha de pagamento — teste)

TesteFolha.java

```
import java.util.ArrayList;

public class TesteFolha {
    public static void main(String[] args) {
        ArrayList<Pagavel> pagaveis = new ArrayList<>();
        pagaveis.add(new Funcionario(3500));
        pagaveis.add(new Freelancer(40, 80));

        double total = 0;
        for (Pagavel p : pagaveis) {
            total += p.calcularPagamento();
        }
        System.out.println("Total a pagar: R$ " + total);
    }
}
```

Saída

Total a pagar: R\$ 6700.0

Resumo do Módulo 4

- **Herança** (`extends`): subclasse ganha atributos/métodos da superclasse.
- ``super(...)`` chama o construtor da base; ``protected`` libera para subclasses.
- **Override** (`@Override`): reescreve o comportamento herdado.
- **Polimorfismo**: tratar tipos diferentes como o tipo comum e chamar seus métodos.
- **Classe abstrata**: não instancia; **método abstrato** obriga subclasses a implementar.
- **Interface** (`implements`): contrato 'o que faz'; uma classe pode ter várias.
- **Object** é a superclasse universal (`toString`, `equals`).

Checkpoint rápido — Módulo 4

1. Palavra-chave da herança:

- (A) implements
- (B) extends
- (C) super

2. Upcasting é:

- (A) Guardar subclasse numa referência da superclasse
- (B) Excluir a superclasse
- (C) Criar um objeto abstrato

3. Interface pode ser instanciada com new?

- (A) Sim
- (B) Não
- (C) Só em Java 21

Gabarito: 1→B, 2→A, 3→B

Módulo 5 · 8 horas

Tratamento de erros e Projeto Final

Código que não quebra + um sistema completo do início ao fim

-
- ✓ Entender e tratar exceções (try/catch)
 - ✓ Lançar exceções com throw
 - ✓ Organizar código em pacotes
 - ✓ Modelar e construir um sistema escolar completo
 - ✓ Testar o sistema e planejar melhorias

Agenda do módulo

1. Exceções

- O que são, try/catch/finally, throw, verificadas x não verificadas

2. Pacotes

- package e import, organização de projetos

3. Projeto final

- Escopo, modelagem, classes, menu interativo, cadastro, busca

4. Testes e encerramento

- Executando, checklist, próximos passos, certificado

O que são exceções?

- **Exceção** = erro que acontece **em tempo de execução** e interrompe o programa.
- Exemplos comuns:
 - `ArithmeticException` — dividir por zero;
 - `NullPointerException` — usar objeto não criado;
 - `InputMismatchException` — digitar texto onde se espera número;
 - `IndexOutOfBoundsException` — acessar posição inexistente da lista.
- Sem tratamento, o programa **morre** e imprime uma pilha de erros (stack trace).
- Com `try/catch`, **capturamos** o erro e continuamos com elegância.

Exemplo: divisão por zero sem tratamento

SemTratamento.java

```
public class SemTratamento {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 0;  
        int resultado = a / b;        // estoura aqui!  
        System.out.println("Resultado: " + resultado);  
        System.out.println("Fim do programa.");  
    }  
}
```

Saída

```
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at SemTratamento.main(SemTratamento.java:5)
```

- A **última linha** nunca executa: o programa foi interrompido.
- A stack trace mostra **onde** o erro aconteceu (arquivo:linha).

Tratando com try/catch

ComTratamento.java

```
import java.util.Scanner;

public class ComTratamento {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        System.out.print("Digite dois números: ");
        int a = teclado.nextInt();
        int b = teclado.nextInt();

        try {
            int resultado = a / b;
            System.out.println("Resultado: " + resultado);
        } catch (ArithmeticException e) {
            System.out.println("Ops! Não dá para dividir por zero.");
        }
        System.out.println("Fim do programa.");
    }
}
```

Saída

```
Digite dois números: 10 0
Ops! Não dá para dividir por zero.
Fim do programa.
```

- O que pode dar errado vai no `try`; a resposta ao erro, no `catch`.
- O programa **continua** após o tratamento — nada de 'morrer'.

try/catch/finally

● ● ● Finally.java

```
public class Finally {  
    public static void main(String[] args) {  
        try {  
            int[] numeros = {1, 2, 3};  
            System.out.println(numeros[5]); // estoura!  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Índice fora dos limites!");  
        } finally {  
            System.out.println("Sempre executo (com erro ou não).");  
        }  
    }  
}
```

Saída

```
Índice fora dos limites!  
Sempre executo (com erro ou não).
```

- **finally** roda **sempre** — usado para fechar recursos (Scanner, arquivos).

Verificadas × não verificadas (e o throw)

- **Não verificadas** (unchecked): `RuntimeException` e filhas — o Java não obriga a tratar.
 - `ArithmeticException`, `NullPointerException`, `InputMismatchException`...
- **Verificadas** (checked): o compilador **exige** try/catch ou `throws`.
 - `IOException`, `SQLException`... (leitura de arquivos, banco).
- **`throw`** lança uma exceção de propósito:
 - `throw new IllegalArgumentException("Idade inválida");`
- É a forma profissional de **recusar valores inválidos** — melhor que imprimir aviso.

Lançando exceções: validando objetos

Pessoa.java

```
public class Pessoa {  
    private String nome;  
    private int idade;  
  
    public void setIdade(int idade) {  
        if (idade < 0) {  
            throw new IllegalArgumentException("Idade não pode ser negativa: " + idade);  
        }  
        this.idade = idade;  
    }  
  
    public void setNome(String nome) {  
        if (nome == null || nome.trim().isEmpty()) {  
            throw new IllegalArgumentException("Nome não pode ser vazio.");  
        }  
        this.nome = nome;  
    }  
}
```

Saída

Erro: Idade não pode ser negativa: -5

- Quem chamar com valor inválido recebe a **exceção com a mensagem clara**.
- No main, envolvemos a chamada em `try/catch` para responder ao usuário.

Pacotes: organizando o projeto

- Com muitos arquivos, organizamos em **pacotes** (pastas lógicas).
- Convenção: domínio invertido — `br.com.escola.modelo`, `br.com.escola.servico`.
- Declara-se no topo: `package br.com.escola.modelo;`
- Para usar uma classe de outro pacote: `import br.com.escola.modelo.Aluno;`
- Classes do **mesmo pacote** não precisam de import.
- Estrutura típica do projeto final:
 - `modelo` → Aluno, Professor, Turma (as classes de dados),
 - `aplicacao` → o programa principal com o menu.

Exemplo: package e import

● ● ● Estrutura.java

```
// arquivo: modelo/Aluno.java
package br.com.escola.modelo;

public class Aluno { ... }

// arquivo: aplicacao/Principal.java
package br.com.escola.aplicacao;

import br.com.escola.modelo.Aluno;
import java.util.Scanner;

public class Principal { ... }
```

- No IntelliJ: botão direito → New → Package, depois New → Java Class.
- Para projetos pequenos, podemos usar o pacote padrão (sem package).

Projeto final: o que vamos construir

- **Sistema Escolar de Gerenciamento** com menu no terminal.
- Funcionalidades:
 - cadastrar aluno (nome, matrícula, nota);
 - cadastrar professor (nome, disciplina);
 - criar turma e **matricular** alunos;
 - listar alunos, professores e turmas;
 - buscar aluno por nome;
 - sair do sistema.
- Reúne tudo: classes, atributos, métodos, encapsulamento, construtores, ArrayList, Scanner, loops.

Projeto final: modelando as classes

- **Aluno:** nome, matricula, nota.
 - métodos: get/set, toString.
- **Professor:** nome, disciplina.
 - métodos: get/set, toString.
- **Turma:** nome, disciplina, ArrayList<Aluno> matriculados.
 - métodos: matricular(Aluno), listarMatriculados(), getNome().
- **Principal:** menu com while, Scanner, ArrayList globais.
- Relacionamento: **Turma** ◆— **Aluno** (composição com lista).

Projeto: classe Aluno

● ● ● Aluno.java

```
public class Aluno {  
    private String nome;  
    private int matricula;  
    private double nota;  
  
    public Aluno(String nome, int matricula, double nota) {  
        this.nome = nome;  
        this.matricula = matricula;  
        this.nota = nota;  
    }  
  
    public String getNome() { return nome; }  
    public int getMatricula() { return matricula; }  
    public double getNota() { return nota; }  
  
    @Override  
    public String toString() {  
        return nome + " (mat. " + matricula + ") - nota " + nota;  
    }  
}
```

Projeto: classe Professor

Professor.java

```
public class Professor {  
    private String nome;  
    private String disciplina;  
  
    public Professor(String nome, String disciplina) {  
        this.nome = nome;  
        this.disciplina = disciplina;  
    }  
  
    public String getNome() { return nome; }  
    public String getDisciplina() { return disciplina; }  
  
    @Override  
    public String toString() {  
        return nome + " - disciplina: " + disciplina;  
    }  
}
```

Projeto: classe Turma

Turma.java

```
import java.util.ArrayList;

public class Turma {
    private String nome;
    private String disciplina;
    private ArrayList<Aluno> matriculados = new ArrayList<>();

    public Turma(String nome, String disciplina) {
        this.nome = nome;
        this.disciplina = disciplina;
    }

    public void matricular(Aluno aluno) {
        matriculados.add(aluno);
    }

    public void listarMatriculados() {
        System.out.println("Turma " + nome + " (" + disciplina + "):");
        if (matriculados.isEmpty()) {
            System.out.println("  (nenhum aluno matriculado)");
        }
        for (Aluno a : matriculados) {
            System.out.println("  - " + a);
        }
    }

    public String getNome() { return nome; }
}
```

Projeto: menu principal (parte 1)

Principal.java

```
import java.util.ArrayList;
import java.util.Scanner;

public class Principal {
    private static ArrayList<Aluno> alunos = new ArrayList<>();
    private static ArrayList<Professor> professores = new ArrayList<>();
    private static ArrayList<Turma> turmas = new ArrayList<>();

    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        int opcao;
        do {
            System.out.println("\n=== SISTEMA ESCOLAR ===");
            System.out.println("1. Cadastrar aluno");
            System.out.println("2. Cadastrar professor");
            System.out.println("3. Criar turma");
            System.out.println("4. Matricular aluno em turma");
            System.out.println("5. Listar tudo");
            System.out.println("0. Sair");
            System.out.print("Opção: ");
            opcao = teclado.nextInt();
            teclado.nextLine(); // consome a quebra de linha
        } while (opcao != 0);
        teclado.close();
    }
}
```

- Estrutura do menu: `do/while` garante que o menu aparece ao menos uma vez.

Projeto: menu principal (parte 2)

Principal.java

```
switch (opcao) {  
    case 1:  
        System.out.print("Nome: ");  
        String nome = teclado.nextLine();  
        System.out.print("Matrícula: ");  
        int mat = teclado.nextInt();  
        System.out.print("Nota: ");  
        double nota = teclado.nextDouble();  
        teclado.nextLine();  
        alunos.add(new Aluno(nome, mat, nota));  
        System.out.println("Aluno cadastrado!");  
        break;  
    // cases 2, 3, 4 seguem o mesmo padrão  
}
```

- O `switch` decide a ação. Variáveis declaradas dentro de cada case.

Projeto: busca de aluno por nome

Principal.java

```
// dentro do case de busca:
System.out.print("Digite um trecho do nome: ");
String trecho = teclado.nextLine();
boolean achou = false;
for (Aluno a : alunos) {
    if (a.getNome().toLowerCase().contains(trecho.toLowerCase())) {
        System.out.println(" - " + a);
        achou = true;
    }
}
if (!achou) {
    System.out.println("Nenhum aluno encontrado.");
}
```

- Mesma técnica da Agenda do Módulo 3 — reutilização de ideias já vistas!

Projeto: como estender (desafios extras)

- Adicionar **exceções**: tratar `InputMismatchException` quando o usuário digitar letra em vez de número.
- Adicionar **validação**: `throw` se o nome for vazio ou a nota for negativa.
- Exibir a **média da turma** (reutilize `calcularMediaDaTurma` do Módulo 3).
- Adicionar **excluir aluno** da lista (`remove` por índice ou por objeto).
- Guardar os dados em **arquivo** (pesquise `BufferedWriter/BufferedReader`).
- Adicionar menu para **atualizar nota** de um aluno.
- Cada item é um novo desafio de POO — tente implementar um por vez!

Projeto: executando o sistema (exemplo)

Uma sessão de uso com cadastro, turma e listagem:

Terminal

```
=== SISTEMA ESCOLAR ===  
1. Cadastrar aluno  
...  
Opção: 1  
Nome: Ana Souza  
Matrícula: 101  
Nota: 8.5  
Aluno cadastrado!  
Opção: 3  
Nome da turma: T1  
Disciplina: P00  
Opção: 5  
ALUNOS:  
- Ana Souza (mat. 101) - nota 8.5  
TURMAS:  
Turma T1 (P00):  
  (nenhum aluno matriculado)
```

- A saída não é gerada por `println` automático — é o resultado das ações do menu.

Checklist de entrega do projeto

- Classe **Aluno** com encapsulamento e `toString`.
- Classe **Professor** com encapsulamento e `toString`.
- Classe **Turma** com `ArrayList<Aluno>` e matrícula.
- **Menu** funcional com as 6 opções e loop correto.
- Cadastro, listagem e busca **testados** com dados reais.
- Programa **não quebra** com entradas erradas (desafio: try/catch).
- Código **legível**: nomes significativos, indentação, sem duplicação.

Como continuar estudando

- **Pratique todos os dias:** 30 minutos de código valem mais que 3 horas só assistindo.
- Recomece o projeto final **sozinho**, sem olhar as soluções.
- Leia código de outras pessoas (GitHub) e refaça à sua maneira.
- Próximos tópicos naturais: coleções ([HashMap](#)), arquivos, banco de dados, interfaces gráficas.
- Recursos oficiais:
 - documentação: docs.oracle.com/javase/tutorial
 - comunidade brasileira: fóruns, Discord, grupos de Java
- **Ferramentas:** aprenda Git + GitHub para versionar seus projetos.
- O importante é **construir** — cada projeto pequeno ensina mais que um curso inteiro.

Resumo final do curso

- **Módulo 0:** Java instalado, primeiros programas, variáveis, fluxo e arrays.
- **Módulo 1:** classes e objetos — o molde e a instância, atributos e métodos.
- **Módulo 2:** encapsulamento, construtores, sobrecarga e static.
- **Módulo 3:** relacionamentos, ArrayList, String e enum.
- **Módulo 4:** herança, polimorfismo, classes abstratas e interfaces.
- **Módulo 5:** exceções, pacotes e um sistema completo construído por você.
- Você saiu do zero e chegou à POO profissional — continue programando! ☕

Verificação final

1. Qual bloco roda SEMPRE, com ou sem erro?

- (A) try
- (B) catch
- (C) finally

2. Como avisamos que um método pode lançar exceção verificada?

- (A) throws
- (B) throw
- (C) try

3. O projeto final usa qual estrutura para repetir o menu?

- (A) if/else
- (B) do/while
- (C) switch sem loop

Gabarito: 1→C, 2→A, 3→B



Você completou as 40 horas!

Do primeiro "Olá, mundo!" a um sistema escolar completo — agora é com você: siga codando!

Bons estudos e ótimos projetos! · Programação Orientada a Objetos com Java

